Review Title Generation: Text Summarization with Encoder-Decoder Models

Omar Belhaj
Université Paris Dauphine-PSL, Master in Artificial Intelligence, Data and Agentic

November 2026

1 Selected task

The task is **abstractive text summarization** in its most extreme form: given an Amazon food review, the model writes a short title of about three words that summarizes the subject and the opinion of the review. The summary is generated word by word and does not need to appear in the review.

Review	*This tea is not good at all. It tastes like nothing and I would not recommend it.*
Generated summary	**not good**

The model estimates the probability of each summary word given the review x and the previous summary words:

$$P(y \mid x) = \prod_{t=1}^{m} P(y_t \mid y_1, \ldots, y_{t-1}, x).$$

Such short summaries help readers scan many reviews quickly, suggest titles when customers leave them empty, and make it easy to count frequent complaints. The whole pipeline is implemented and trained from scratch in PyTorch, with no pretrained generative model.

2 Dataset

The data is **Amazon Fine Food Reviews** (McAuley & Leskovec, 2013). The input is the review text and the target is the title written by the same author, which serves as a reference summary. Star ratings are used only for evaluation.

- **Cleaning.** From 568,454 reviews, removing duplicates leaves 393,552 and length filters leave 317,758. Reviews keep 10–200 tokens and are truncated to 80; titles keep 2–10 tokens.
- **Split by product.** 60,000 reviews are sampled and split about 80/10/10 so that test products are never seen in training: 47,762 train, 6,137 validation, 6,101 test. The final model uses 150,000 training reviews, still excluding validation and test products.
- **Preprocessing.** Lowercasing, HTML removal and a tokenizer that keeps contractions such as *don't*, which preserves negation. Vocabularies are built on train only, with words seen at least 3 times: 16,429 review words and 4,342 title words, including `<PAD>`, `<UNK>`, `<SOS>` and `<EOS>`.
- **Two difficulties.** 78.9% of test reviews are positive, and generic titles are frequent: *great product* is the most common title.

3 Architecture

Three models change one design decision at a time, then a final model adds improvements based on the error analysis.

- **Encoder.** Embeddings of dimension 128 feed an LSTM with 256-dimensional states (a bidirectional LSTM uses 128 units per direction). It returns one state h_i per review word. A linear bridge with tanh maps its final state to the decoder's initial state.
- **Decoder.** An LSTM of 256 units generates one word at a time from `<SOS>` until `<EOS>`, through a linear layer and a softmax over the title vocabulary.

- **Bahdanau attention.** At each step, the decoder weights every encoder state, so it is not limited to one fixed-size vector:

$$e_{t,i} = v^\top \tanh(W_s s_{t-1} + W_h h_i), \quad \alpha_{t,i} = \mathrm{softmax}_i(e_{t,i}), \quad c_t = \textstyle\sum_i \alpha_{t,i} h_i.$$

- **Pointer-generator** (final model, See et al., 2017). A learned probability p_{gen} mixes generating a word with copying a review word, which helps with product names outside the vocabulary:

$$P(w) = p_{gen} P_{vocab}(w) + (1 - p_{gen}) \textstyle\sum_{i:x_i=w} \alpha_{t,i}.$$

4 Training procedure

All models minimise token-level cross-entropy with Adam (learning rate 0.001, halved when validation loss stalls), batches of 128, dropout 0.3, gradient clipping at 1, and up to 15 epochs with early stopping after 3 epochs without improvement. **Teacher forcing** is used with a ratio of 1: during training, the decoder receives the true previous word. Generation uses greedy decoding; beam search with 5 beams was also tested. Training takes 12 to 25 seconds per epoch on a T4 GPU for the first models, and about 84 seconds for the final model.

The attention models start to overfit after about epoch 8 (Figure 1). The baseline reached the 15-epoch limit, so it was retrained with up to 30 epochs: its test ROUGE-1 only moved from 0.105 to 0.106, so the comparison is fair.

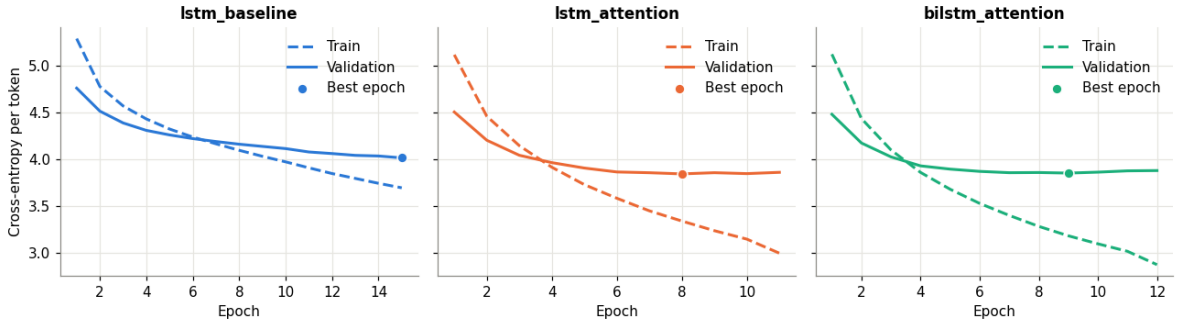


Figure 1: Training and validation loss of the three architectures.

5 Results

ROUGE is the main metric for summarization. Since overlap alone can be misleading, the evaluation also measures diversity and whether the summary keeps the sentiment of the star rating (balanced accuracy of a small sentiment classifier; human titles score 0.862). The final model was chosen on validation ROUGE-1, a rule fixed in advance.

Model (test set)	ROUGE-1	ROUGE-2	BLEU-2	Distinct titles	Sentiment acc.
Most frequent title (no learning)	0.066	0.011	1.26	0.0%	0.500
First 4 words of the review (no learning)	0.094	0.028	4.84	85.9%	0.635
LSTM baseline	0.105	0.021	3.03	0.5%	0.822
LSTM + attention	0.153	0.042	5.76	16.7%	0.753
BiLSTM + attention	0.159	0.043	6.11	23.1%	0.754
+ balanced sampling	0.148	0.038	5.33	24.7%	0.783
+ copy mechanism	0.163	0.047	6.26	36.4%	0.725
+ 150,000 training reviews	0.180	0.054	7.25	34.7%	0.809
Final model: all three combined	**0.183**	**0.057**	**7.40**	**41.9%**	**0.830**

Table 1: Test results. Paired bootstrap intervals (95%) on ROUGE-1: attention adds +0.048 [+0.043, +0.053]; the final model adds +0.025 [+0.019, +0.030] over BiLSTM + attention.

- **Attention** is the architecture choice with the clearest effect. The baseline barely beats copying the first four words and outputs *great product* for 45.6% of reviews.
- **More data** is the strongest single improvement and delays overfitting (best epoch 14 instead of 9).

- **The bidirectional encoder, beam search and copying alone** bring no clearly established ROUGE-1 gain. Balanced sampling alone improves sentiment but lowers ROUGE-1.
- **Manual check.** Of 50 random test summaries from a second run of the final model, 40 were judged acceptable (10 of them generic), against a mean ROUGE-1 of 0.198. 23 acceptable summaries have a ROUGE-1 of 0, which shows that ROUGE underestimates quality for such short summaries.

6 Generated examples

Review (shortened)	BiLSTM + attention	Final model	Verdict
Dog food in a broken bag, my dog refused to eat it	dog food	dog hated it	Fixed
This tea is not good at all	not as good as the tea	not good	Fixed
Flavor is great, but the price is far too high	good but not great	good flavor but too high	Fixed
It's okay. Not bad, not great, an average cereal	not as good as i remember	it's okay	Fixed
Expected chips to be terrible, but really tasty	not bad	terrible taste	Wrong

Table 2: Hand-written reviews before and after the improvements. These are illustrations, not statistics.

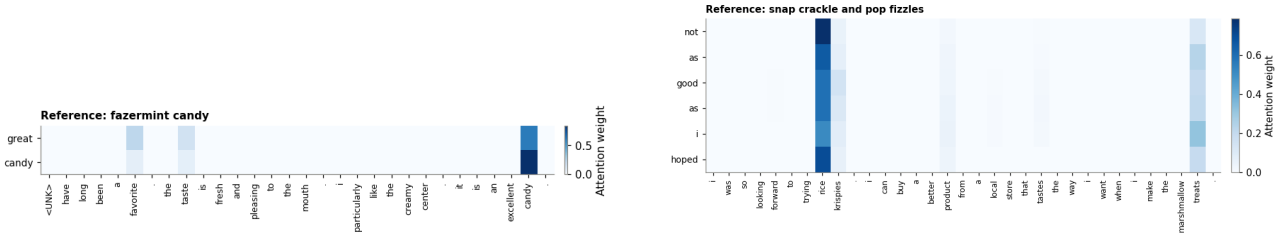


Figure 2: Attention maps of BiLSTM + attention. Left: *candy* is generated while attention sits on *candy*. Right: attention stays on *rice* while the decoder writes a stock phrase.

Successes show that the model captures the subject and the opinion, often with its own wording (*best canned tuna* for *great canned tuna*). Failures include reversed sentiment (*dog loves it* for *dog won't eat it*, which still scores 0.50 in ROUGE-1), repetition (*gift was a gift gift*) and generic summaries.

7 Limitations and possible improvements

Limitation	Evidence	Possible improvement
Contrast and structure	*terrible taste* for a tasty product	More data, a Transformer encoder
Attention without memory	Repetitions such as *honey honey*	Coverage mechanism
Generic summaries	The top summary is still 9.9% of outputs	All eligible reviews, pretrained embeddings
Weak on negative reviews	25.5% of negative reviews get a positive summary	Tune the sampling weights
Overlap metrics	ROUGE misses meaning in both directions	Larger manual evaluation
One run per model	Small differences are not established	Several random seeds

Conclusion. The from-scratch encoder–decoder learns to summarize reviews in a few words. Attention and more training data bring the main gains, reaching a ROUGE-1 of 0.183 and 40 acceptable summaries out of 50. The remaining errors come mostly from sentence structure and data imbalance rather than from model size.

References. Bahdanau et al. (2015), *Neural machine translation by jointly learning to align and translate*, ICLR. McAuley & Leskovec (2013), *From amateurs to connoisseurs*, WWW. See et al. (2017), *Get to the point: Summarization with pointer-generator networks*, ACL. Lin (2004), *ROUGE*, ACL Workshop.